

# Incoming Request Processing Workflow

## Maintenance Document

Authored by Yosar Fatahillah

*AI-assisted triage and remediation for customer operations teams*

### Table of Contents

- 1. Introduction.....2
  - Intended Audience.....2
- 2. Technology Stack.....2
- 3. System Architecture Overview.....2
  - Data Flow.....2
  - Case Status Lifecycle.....3
  - Repository Structure.....3
- 4. Features - Technical Reference.....3
  - Feature 1 - Process a Request.....3
  - Feature 2 - Batch Processing.....4
  - Feature 3 - Review Queue (Human-in-the-Loop).....4
  - Feature 4 - Dashboard & Audit Log.....4
  - Feature 5 - Admin (Stakeholder Directory & Outbox).....5
- 5. Development Environment Setup.....5
  - 5.1 Prerequisites.....5
  - 5.2 Local Setup.....5
  - 5.3 Running.....5
  - 5.4 Resetting Demo Data.....6
- 6. Data Management.....6
  - 6.1 Data Stores Overview.....6
  - 6.2 Schema.....6
  - 6.3 Sample Dataset & Licensing.....6
- 7. Security Protocols & Measures.....6
- 8. Adding New Features.....7
  - 8.1 Adding a New Remediation Branch.....7
  - 8.2 Adding a New Notification Channel.....7
  - 8.3 Adding a New Stakeholder Position Type.....7
- 9. Known Limitations.....7
- 10. Troubleshooting - Developer Reference.....8

## 1. Introduction

This Maintenance Document is intended for developers responsible for maintaining, extending, or modifying the Incoming Request Processing Workflow prototype after handover. It covers the actual technology stack, architecture, all implemented features, guidance on extending the codebase, data management, security, and troubleshooting.

### Intended Audience

Python developers with working knowledge of Streamlit, SQLite, and REST-style API integration. Comfortable running scripts from a terminal. Read alongside the README (setup tutorial) and the Product Document (business context).

## 2. Technology Stack

| Layer           | Tool / Technology             | Version & Role                                                                         |
|-----------------|-------------------------------|----------------------------------------------------------------------------------------|
| Language        | Python 3                      | 3.10+ - all orchestration and workflow logic                                           |
| AI              | Anthropic Claude API          | model claude-sonnet-4-5 - classification and draft generation, via the anthropic SDK   |
| UI              | Streamlit                     | 1.38+ - five-tab prototype interface                                                   |
| Charting        | Altair                        | bundled with Streamlit - labelled bar charts on the dashboard                          |
| Data            | pandas                        | 2.2+ - dashboard aggregation, CSV export                                               |
| Storage         | SQLite (stdlib sqlite3)       | case log, stakeholder directory, notification outbox - single file at data/case_log.db |
| Email           | smtplib (stdlib) + Gmail SMTP | stakeholder notifications; requires a Gmail App Password                               |
| Resilience      | Built-in regex engine         | deterministic fallback classifier when the Claude API is unavailable                   |
| Version Control | GitHub                        | github.com/yos-fatahillah/incoming-request-workflow                                    |

## 3. System Architecture Overview

The prototype is a single-process Streamlit application backed by a local SQLite database. There is no separate backend service; classifier.py and notifier.py call external APIs (Anthropic, Gmail SMTP) directly from within the Streamlit process.

### Data Flow

Intake (Streamlit UI or CLI batch runner)

- > classifier.py (Claude API classification; offline fallback on failure)
- > workflows.py (confidence-gated override check, then branch handler)
  - > notifier.py (stakeholder email; simulated outbox on failure)
- > storage.py (SQLite: case log, status lifecycle, stakeholder directory)
- > Dashboard tab (pandas + Altair aggregation, ROI calculator)

### Case Status Lifecycle

| Status          | Meaning                                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------|
| RESOLVED        | Fully auto-resolved by the workflow (General Enquiry branch only).                                                          |
| IN_PROGRESS     | Triaged and routed by the AI; the actual task or resolution is owned by a human team (Complaint, Service Request branches). |
| HELD_FOR_REVIEW | Escalation branch or low-confidence override; awaiting a supervisor decision in the Review Queue.                           |
| APPROVED        | A supervisor reviewed a held case and approved the AI's handling.                                                           |

## Repository Structure

| File                             | Role                                                                                                                                                 |
|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| app.py                           | Streamlit UI - five tabs: Process a Request, Batch Processing, Review Queue, Dashboard & Audit Log, Admin.                                           |
| classifier.py                    | Claude API classification plus the offline keyword fallback. Returns request_type, urgency, sub_topic, confidence, key_details, and the engine used. |
| workflows.py                     | The four branch handlers, the confidence-gated override, draft message generation, and department routing lookups.                                   |
| storage.py                       | SQLite persistence: case log with status lifecycle, stakeholder directory (seeded with defaults), and dashboard summary aggregation.                 |
| notifier.py                      | Stakeholder email notifications via Gmail SMTP, with a simulated-outbox fallback. Both paths log to the same outbox table.                           |
| sample_data.py                   | Ten synthetic sample requests (AI-generated, no real data) covering all four branches plus one deliberately ambiguous edge case.                     |
| run_batch_demo.py                | CLI batch runner - processes the full sample set and prints a readable trace. Useful for smoke testing without the UI.                               |
| requirements.txt<br>.env.example | / Dependency pins and an API key setup template.                                                                                                     |

## 4. Features - Technical Reference

The prototype comprises five tabs. Each section below lists the key files, data sources, and guidance for modification.

### Feature 1 - Process a Request

| Component          | Detail                                                                                                                                             |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| Screen             | app.py, Tab 1 - three intake modes: simulated inbox picker, manual text entry, .txt file upload                                                    |
| Classification     | classifier.classify_request() - calls Claude API, falls back to offline keyword engine on any failure                                              |
| Workflow execution | workflows.run_workflow() - confidence check, then dispatch to the matching branch handler                                                          |
| Logging            | storage.log_case() - every processed request, regardless of source, is written to the case log                                                     |
| To extend          | Add a new intake mode (e.g. email polling) by producing plain request text and calling process_and_log() the same way the existing three modes do. |

### Feature 2 - Batch Processing

| Component      | Detail                                                                                                                         |
|----------------|--------------------------------------------------------------------------------------------------------------------------------|
| Screen         | app.py, Tab 2 - one-click run of the full sample_data.SAMPLE_REQUESTS list                                                     |
| CLI equivalent | run_batch_demo.py - same logic, terminal output, no Streamlit dependency                                                       |
| To extend      | Add more entries to SAMPLE_REQUESTS (id, expected_branch, text) to widen batch/demo coverage. No code change needed elsewhere. |

### Feature 3 - Review Queue (Human-in-the-Loop)

| Component         | Detail                                                                                                                               |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Screen            | app.py, Tab 3 - lists cases with status HELD_FOR_REVIEW                                                                              |
| Query             | storage.fetch_review_queue() - filters the case table by status                                                                      |
| Approve action    | storage.update_case_status(request_id, 'APPROVED')                                                                                   |
| Reclassify action | Re-runs the chosen branch handler from workflows.BRANCH_MAP and overwrites the case via storage.update_case_after_reclassify()       |
| To extend         | Add an audit note field to capture why a supervisor reclassified a case, for the feedback-loop roadmap item (see Known Limitations). |

### Feature 4 - Dashboard & Audit Log

| Component      | Detail                                                                                                                                                                                        |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Screen         | app.py, Tab 4 - headline metrics, three labelled Altair charts, ROI calculator, filterable audit table with CSV export                                                                        |
| Summary data   | storage.summary_counts() - volumes by type/urgency/status/branch, override count, deflection percentage                                                                                       |
| ROI calculator | Branch-weighted: minutes saved = sum over cases of (branch default minutes x case count); cost = minutes / 60 x hourly rate. All five assumptions are editable in an expander, not hardcoded. |
| Chart labels   | app.labeled_bar_chart() - Altair chart with a mark_text layer, since st.bar_chart cannot render data labels                                                                                   |
| To extend      | Add a time-series view (volume per day) once cases carry more than a single day of demo data - the timestamp column already supports this.                                                    |

### Feature 5 - Admin (Stakeholder Directory & Outbox)

| Component      | Detail                                                                                                                                            |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Screen         | app.py, Tab 5 - editable stakeholder table (st.data_editor) and the notification outbox                                                           |
| Storage        | storage.fetch_stakeholders() / replace_stakeholders() - full-table replace on Save, seeded with DEFAULT_STAKEHOLDERS on first run                 |
| Routing lookup | storage.find_stakeholder(position_contains) - case-insensitive substring match against the position field, called from workflows.py               |
| Outbox         | notifier.fetch_outbox() - every notification, real or simulated, with a delivery column showing 'sent' or 'simulated'                             |
| To extend      | Add a department field distinct from position if routing needs to support multiple people per department rather than one match-first stakeholder. |

## 5. Development Environment Setup

### 5.1 Prerequisites

| Tool                                   | Version / Notes                                                                    |
|----------------------------------------|------------------------------------------------------------------------------------|
| Python                                 | 3.10 or later                                                                      |
| pip                                    | bundled with Python; on macOS use python3 -m pip if the pip command is not on PATH |
| Anthropic API key                      | optional - console.anthropic.com; required for live classification & drafting      |
| Gmail account with 2-Step Verification | optional - required only to send real stakeholder emails via an App Password       |

### 5.2 Local Setup

```
git clone https://github.com/yos-fatahillah/incoming-request-workflow.git
cd incoming-request-workflow
pip install -r requirements.txt
```

Runtime configuration is via environment variables, never committed source files:

| Variable                  | Purpose                                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------|
| ANTHROPIC_API_KEY         | enables live Claude classification and drafting; offline fallback used if unset            |
| SMTP_USER / SMTP_PASSWORD | Gmail address and App Password for real stakeholder emails; simulated outbox used if unset |
| SMTP_HOST / SMTP_PORT     | override the default smtp.gmail.com : 465 if using a non-Gmail provider                    |

*SECURITY - never commit real values. .env.example is a blank template; a real .env (or exported shell variables) should never be added to version control. The project .gitignore excludes .env and the local database.*

### 5.3 Running

```
streamlit run app.py          # full UI at http://localhost:8501
python3 run_batch_demo.py     # CLI batch smoke test, no UI
```

### 5.4 Resetting Demo Data

Stop the app and delete data/case\_log.db. It is recreated automatically, with the default stakeholder directory reseeded, on the next launch. Do this after any schema change or when stale historical rows would make dashboard numbers misleading.

## 6. Data Management

### 6.1 Data Stores Overview

A single SQLite file (data/case\_log.db) holds three tables: cases (the audit trail), stakeholders (the admin-editable routing directory), and outbox (every notification sent or simulated). There is no external database - this is a single-user local prototype, not a shared deployment.

### 6.2 Schema

| Table        | Key Fields                                                                                                                                                         | Notes                                                                                                    |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| cases        | request_id, timestamp, raw_text, request_type, urgency, sub_topic, confidence, engine, branch, override_applied, override_reason, steps_json, outputs_json, status | steps_json and outputs_json store the full per-case remediation trace as JSON text                       |
| stakeholders | id, name, position, email                                                                                                                                          | position is matched by keyword (e.g. 'Supervisor', 'Department Owner - Billing') from workflows.py       |
| outbox       | id, timestamp, recipient_name, recipient_email, subject, body, delivery                                                                                            | delivery is 'sent' for a real email or 'simulated (...)' with the failure reason if SMTP was unavailable |

### 6.3 Sample Dataset & Licensing

The ten sample requests in sample\_data.py are AI-generated for this prototype; no proprietary or real customer data is used anywhere in the build. No third-party licensed datasets are involved.

## 7. Security Protocols & Measures

- No credentials are stored in source files. ANTHROPIC\_API\_KEY, SMTP\_USER, and SMTP\_PASSWORD are read exclusively from environment variables at runtime.
- The Gmail integration uses an App Password, a revocable machine-only credential, never the account's primary password.
- The local SQLite database is not exposed over any network interface; it is a plain file on the machine running the app.
- Request text sent to the Anthropic API is not persisted by this application beyond the local audit trail; retention on the API provider's side follows Anthropic's own data policy.
- Customer-facing messages are never auto-sent for the Escalation branch; drafts are held for human review by design, not by configuration that could be disabled.

*Before any production or shared deployment: move secrets to a managed secret store, add authentication and role separation (currently single-user), and migrate from SQLite to a networked database such as PostgreSQL.*

## 8. Adding New Features

### 8.1 Adding a New Remediation Branch

1. Write a new handle\_<branch>(request\_id, text, classification) function in workflows.py, following the pattern of the existing four handlers (steps list + outputs dict).
2. Register it in BRANCH\_MAP so run\_workflow() and the Review Queue's reclassify dropdown can find it.
3. Add a default case status for the branch in storage.BRANCH\_DEFAULT\_STATUS.
4. Add at least one sample request with that expected branch to sample\_data.py for batch-run coverage.

### 8.2 Adding a New Notification Channel

1. Add a new send function in notifier.py alongside notify(), following the same try-real-then-fall-back-to-outbox pattern.
2. Call it from the relevant branch handler in workflows.py in place of, or alongside, the existing notify() call.
3. Extend the outbox table (or add a channel column) in storage.py if the Admin tab should distinguish channels in its outbox view.

### 8.3 Adding a New Stakeholder Position Type

1. Add a row to storage.DEFAULT\_STAKEHOLDERS with a distinctive position string.
2. Call `storage.find_stakeholder('your position keyword')` from the relevant branch handler in `workflows.py`.
3. No schema change needed - the stakeholders table already supports arbitrary position strings.

## 9. Known Limitations

| Limitation                                    | Detail & Recommended Action                                                                                                                                             |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No accuracy evaluation                        | The 10-message sample set demonstrates the pipeline but does not measure it. Build a labelled test set (~100 messages) with a per-category confusion matrix.            |
| Urgency is displayed, not consumed            | A High-urgency service request gets the same 24h SLA as a Medium one. Urgency should scale the SLA window, follow-up timing, and notification priority.                 |
| Keyword department routing                    | Fast and explainable, but coarser than letting the model choose the department directly. Keep the keyword rules as a fallback if this is upgraded.                      |
| No few-shot examples in the classifier prompt | Category definitions only. Adding 2-3 worked examples per category would tighten consistency on vague or short messages.                                                |
| Multi-intent messages take one branch         | A message containing both a complaint and an enquiry rides the complaint path only; the enquiry half is not split out.                                                  |
| No case threading                             | Each message is a new case. A customer identifier would allow linking repeat contacts and auto-escalating on a pattern.                                                 |
| No feedback loop                              | Human reclassifications are logged (engine = human-reclassified) but nothing yet learns from them. This is the natural next step since the data is already captured.    |
| Single-user, local database                   | SQLite with no authentication. Needs a networked database and access control before any shared or production deployment.                                                |
| No automated tests                            | Correctness currently relies on manual testing. A pytest suite over the classifier fallback, override logic, and status transitions is the next reliability investment. |

## 10. Troubleshooting - Developer Reference

| Issue                                                            | Likely Cause                                                                                                             | Resolution                                                                                                                                                |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| pip / streamlit: command not found (macOS)                       | Python installed but its scripts directory isn't on PATH                                                                 | Use <code>python3 -m pip install -r requirements.txt</code> and <code>python3 -m streamlit run app.py</code> instead of the bare commands                 |
| Classifier always uses the offline engine even with a key set    | ANTHROPIC_API_KEY not actually exported in the current terminal session, or contains a placeholder instead of a real key | Run <code>echo "[\$ANTHROPIC_API_KEY]"</code> to inspect the value; re-export with the full key copied directly from the console                          |
| 401 invalid x-api-key                                            | The exported key string is wrong, truncated, or a placeholder                                                            | Re-copy the full key from <code>console.anthropic.com</code> -> API Keys (shown once at creation); revoke and recreate if the original was never saved    |
| Classification succeeds but drafts read as generic template text | API calls are failing silently and falling back offline                                                                  | Check the case trace's Engine field for the specific error name; usually a billing/credit issue -- verify a non-zero balance under Billing in the console |

| Issue                                                                     | Likely Cause                                                                                                     | Resolution                                                                                                                                                                                                                   |
|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Notifications always show 'simulated' in the Outbox                       | SMTP_USER / SMTP_PASSWORD not set, or using the Gmail account password instead of an App Password                | Generate an App Password at <a href="https://myaccount.google.com/apppasswords">myaccount.google.com/apppasswords</a> (requires 2-Step Verification) and export both variables in the same terminal session before launching |
| Environment variables 'disappear' the next day                            | export only persists for the current terminal session                                                            | Re-export at the start of every session, or keep them in a private file outside the repo and source it before launching                                                                                                      |
| Dashboard numbers look inconsistent (e.g. many IN_PROGRESS, few RESOLVED) | Stale rows from before the status column existed were backfilled with a default status during a schema migration | Stop the app, delete data/case_log.db, relaunch, and reprocess -- this clears legacy rows and recomputes cleanly                                                                                                             |
| Streamlit app boots but shows a blank or errored page                     | A syntax error was introduced in app.py, or a required package is missing                                        | Check the terminal running Streamlit for a traceback; confirm pip install -r requirements.txt completed without errors                                                                                                       |

## 11. Handover Note

This Maintenance Document is part of the submission package for the Incoming Request Processing Workflow prototype. It should be read alongside the Product Document (business context) and the README (quick-start tutorial).

| Asset                        | Location                                                                                                                      |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Source code & README         | <a href="https://github.com/yos-fatahillah/incoming-request-workflow">github.com/yos-fatahillah/incoming-request-workflow</a> |
| Product Document             | Included in submission package                                                                                                |
| Product Requirement Document | Included in submission package                                                                                                |
| Five-slide summary deck      | Included in submission package (PDF & PPT)                                                                                    |
| Demo recording               | Included in submission package                                                                                                |